Vehicle Parking App — Project Report

Author

Rohit Singh

Roll Number: 24f2005749

Email: 24f2005749@ds.study.iitm.ac.in

I am a student of BS Data Science (diploma). I am a tech-enjoyer and really love to learn new skills.

Description

This project is about a vehicle parking app using Flask and SQLAlchemy. This app allows admins to manage their parking lots, spots, and users efficiently as well as provide users with various options for parking that too easily.

- Authentication (role-based control): ~ 2%
- Debugging : ~ 3%
- Jinja templating: ~ 1%
- Route handling: ~ 7%
- Bootstrap / CSS: ~5%
- Charts / Data Visualization: ~3%

Technologies Used

- Flask – Easy to learn and implement backend and routing.
- SQLAlchemy – Provides ORM-based interaction with the database.
- Bootstrap – Reduces the styling work and makes frontend development efficient.
- Jinja2 Templates – for dynamic HTML rendering
- Chart.js – Creates interactive graphs for admin dashboard.
- Werkzeug – Adds security and password handling functionality.

DB Schema Design

1. User

- u_id – Primary Key (Integer)
- username – Unique, Not Null (String)
- passhash – Not Null (String)
- u_name – Not Null (String)
- u_add – Not Null (String)
- u_pin – Not Null (Integer)
  Relationships: One-to-many with Reservation

2. Admin

- adm_id – Primary Key (Integer)
- adm_username – Unique, Not Null (String)
- adm_passhash – Not Null (String)
- adm_name – Nullable (String)

3. ParkingLot

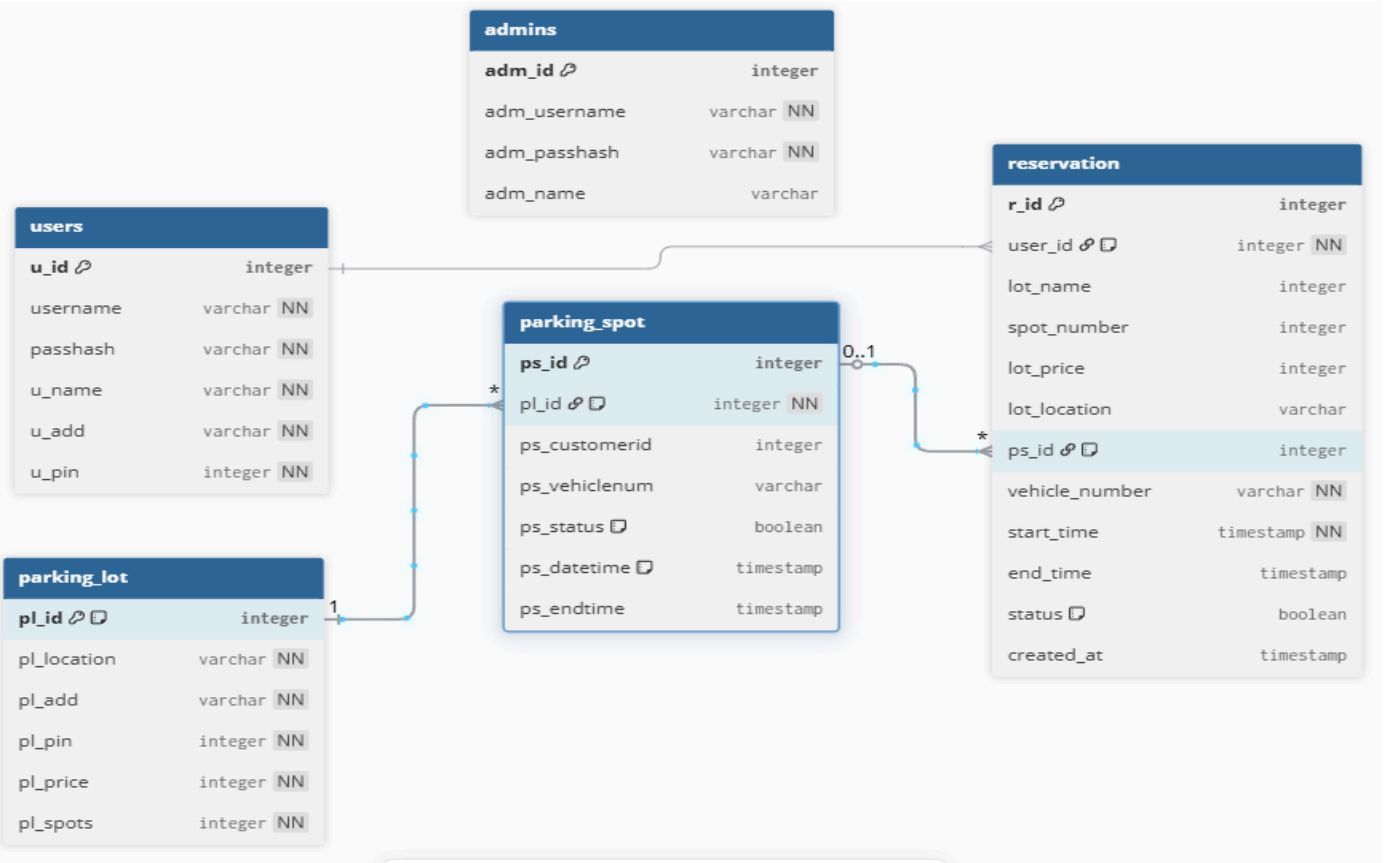
- pl_id – Primary Key (Integer, Auto-increment)
- pl_location – Not Null (String)
- pl_add – Not Null (String)
- pl_pin – Not Null (Integer)
- pl_price – Not Null (Integer)
- pl_spots – Not Null (Integer)
  Relationships: One-to-many with ParkingSpot (with cascade delete)

4. ParkingSpot

- ps_id – Primary Key (Integer)
- pl_id – Foreign Key referencing ParkingLot.pl_id, Not Null, Cascade on delete
- ps_customerid – Nullable (Integer)
- ps_vehiclenum – Nullable (String)
- ps_status – Defaults to False (Boolean)
- ps_datetime – Defaults to ist_now() (Datetime, IST adjusted)
- ps_endtime – Nullable (Datetime)
  Relationships: One-to-one (or many-to-one) with Reservation

5. Reservation

- r_id – Primary Key (Integer)
- user_id – Foreign Key referencing User.u_id (Not Null)
- lot_name – Not Null (Integer)
- spot_number – Not Null (Integer)
- lot_price – Not Null (Integer)
- lot_location – Not Null (String)
- ps_id – Foreign Key referencing ParkingSpot.ps_id (Nullable, SET NULL on delete)
- vehicle_number – Not Null (String)
- start_time – Defaults to ist_now() (Datetime, Not Null)
- end_time – Nullable (Datetime)
- status – Defaults to True (Boolean)
- created_at – Defaults to ist_now() (Datetime)



Design Reasoning:
The schema ensures efficient storage, with foreign keys maintaining referential integrity. Cascade deletions prevent excess elements. The datetime is timezone-adjusted for IST.

Architecture and Features

The project follows a structured MVC pattern:
- Controllers (Routes): Flask routes are defined in routes.py
- Templates: All frontend pages are placed under templates/ using Jinja2 templating.
- Static Assets: CSS, JS, and image files are stored in the static/ directory.
- Database Models: Defined using SQLAlchemy with relationships and constraints.

Implemented Features:
- Admin login and control panel
- Add/manage parking lots and parking spots
- Real-time tracking of active/inactive reservations
- Interactive bar charts for usage summary
- User dashboard for making and releasing reservations
- Automatic expiration and status update of booked spots

Video

Watch the video demo here: Project